Enrollment No : 240840116062

Practical 5 : Analyze datasets using correlation, covariance, outlier detection, and distribution analysis

Step1:Import libraries

```
# import  libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step2: Create the dataset and print data frame **

```
# define student dataset and create  dataframe
data = {
    'student_Id':[101,102,103,104,105,106,107,108,109,110,111,112,113,114,115],
    'study_hours':[2,3,4,5,6,7,8,9,3,4,5,6,7,10,1],
    'attendance':[65,70,75,80,82,85,90,92,68,74,78,84,88,95,40],
    'assignment_score':[55,60,65,70,72,78,82,88,62,67,71,76,80,95,35],
    'Final_marks':[50,55,62,68,72,78,82,90,58,63,69,75,81,98,25]
}

df = pd.DataFrame(data)
df
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
0	101	2	65	55	50
1	102	3	70	60	55
2	103	4	75	65	62
3	104	5	80	70	68
4	105	6	82	72	72
5	106	7	85	78	78
6	107	8	90	82	82
7	108	9	92	88	90
8	109	3	68	62	58
9	110	4	74	67	63
10	111	5	78	71	69
11	112	6	84	76	75
12	113	7	88	80	81
13	114	10	95	95	98
14	115	1	40	35	25

Next steps: Generate code with df | New interactive sheet

Step3: Inspect the Dataset.

Task 3.1: Display first five records from the dataset .

```
# display first five records of dataset
df.head()
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
0	101	2	65	55	50
1	102	3	70	60	55
2	103	4	75	65	62
3	104	5	80	70	68
4	105	6	82	72	72

Next steps: Generate code with df | New interactive sheet

Task 3.2: Display information of the dataset with the null value.

```
# display structural information of dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   student_Id        15 non-null     int64
 1   study_hours       15 non-null     int64
 2   attendance        15 non-null     int64
 3   assignment_score  15 non-null     int64
 4   Final_marks       15 non-null     int64
dtypes: int64(5)
memory usage: 732.0 bytes
```

Task 3.3 : Display the number of rows And columns.

```
# Display shape  of dataframe
df.shape
```

```
(15, 5)
```

Task 3.4 : Generating the basic descriptive Statistics.

```
# generate basic descriptive statistics
df.describe()
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
count	15.000000	15.000000	15.000000	15.000000	15.000000
mean	108.000000	5.333333	77.733333	70.400000	68.400000
std	4.472136	2.581989	13.718948	14.500246	17.759504
min	101.000000	1.000000	40.000000	35.000000	25.000000
25%	104.500000	3.500000	72.000000	63.500000	60.000000
50%	108.000000	5.000000	80.000000	71.000000	69.000000
75%	111.500000	7.000000	86.500000	79.000000	79.500000
max	115.000000	10.000000	95.000000	95.000000	98.000000

Step 4: Calculate Correlation.

Task 4.1 : Calculate the correlation between study hours and Final marks.

```
# calculate correlation between study hours and final marks
correlation = df['study_hours'].corr(df['Final_marks'])
print(correlation)
```

```
0.9626650934516625
```

Task 4.2: Display the correlation.

```
# calculate  correlation of all columns
df.corr()
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	1.000000e+00	0.247436	-7.279626e-16	0.136585	0.128606
study_hours	2.474358e-01	1.000000	9.201931e-01	0.961552	0.962665
attendance	-7.279626e-16	0.920193	1.000000e+00	0.981905	0.983763
assignment_score	1.365851e-01	0.961552	9.819053e-01	1.000000	0.999268
Final_marks	1.286063e-01	0.962665	9.837631e-01	0.999268	1.000000

Step 5: Calculate Correlation Matrix.

Task 5.1: Calculate correlation among all numerical variables.

```
# Show and create a correlation matrix for all columns
correlation_matrix = df.corr()
correlation_matrix
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	1.000000e+00	0.247436	-7.279626e-16	0.136585	0.128606
study_hours	2.474358e-01	1.000000	9.201931e-01	0.961552	0.962665
attendance	-7.279626e-16	0.920193	1.000000e+00	0.981905	0.983763
assignment_score	1.365851e-01	0.961552	9.819053e-01	1.000000	0.999268
Final_marks	1.286063e-01	0.962665	9.837631e-01	0.999268	1.000000

Next steps: Generate code with `correlation_matrix` New interactive sheet

Task 5.2 : Display the correlation Matrix.

```
# display correlation matrix
correlation_matrix = df.corr()
correlation_matrix
```

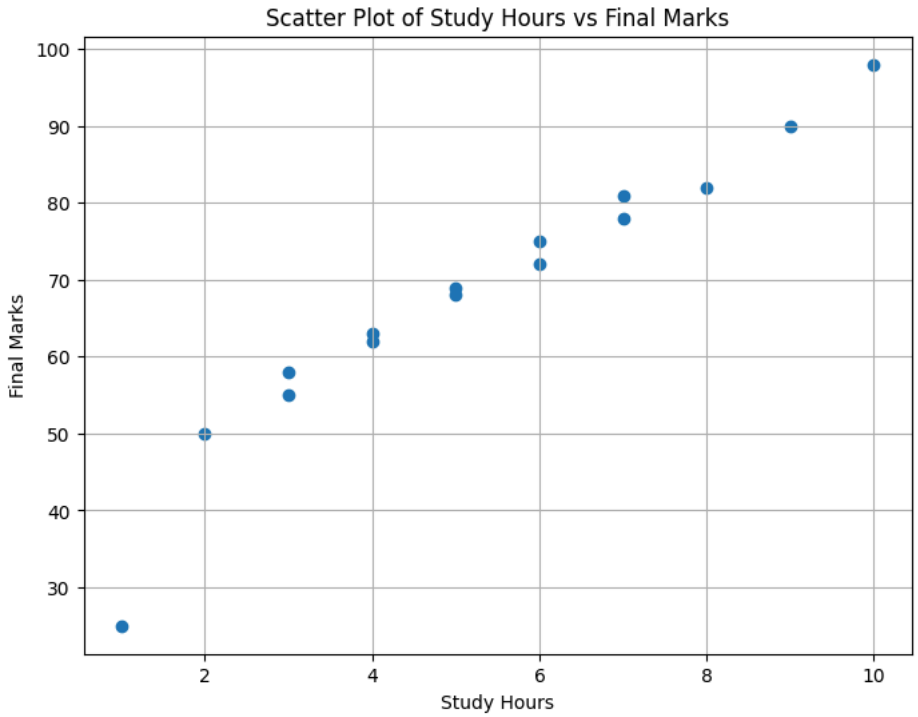
	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	1.000000e+00	0.247436	-7.279626e-16	0.136585	0.128606
study_hours	2.474358e-01	1.000000	9.201931e-01	0.961552	0.962665
attendance	-7.279626e-16	0.920193	1.000000e+00	0.981905	0.983763
assignment_score	1.365851e-01	0.961552	9.819053e-01	1.000000	0.999268
Final_marks	1.286063e-01	0.962665	9.837631e-01	0.999268	1.000000

Next steps: Generate code with `correlation_matrix` New interactive sheet

Step 6 : Visualize Correlation Using Scatter Plot.

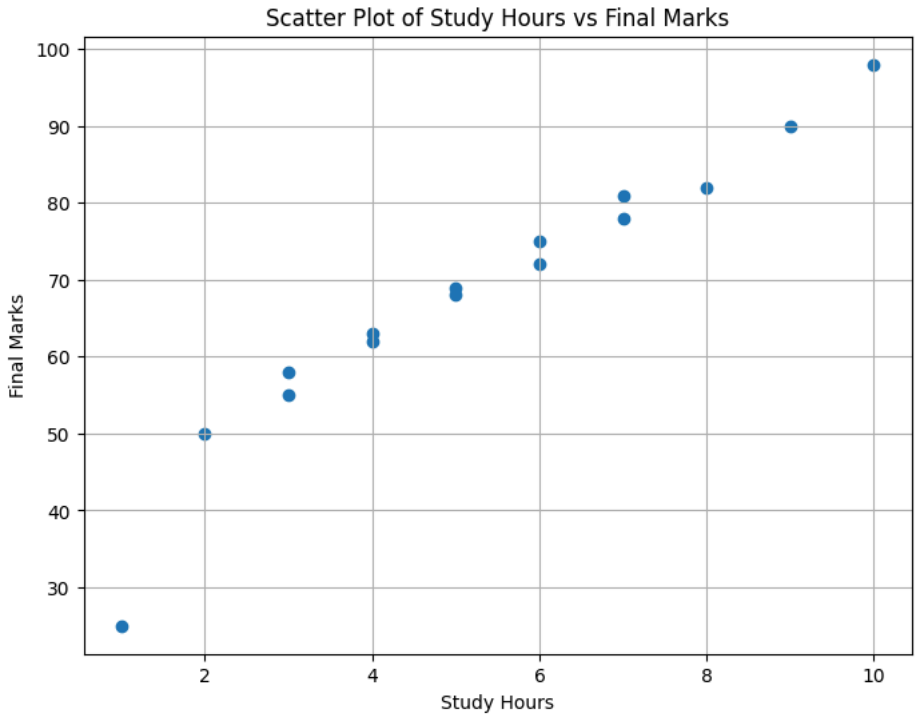
Task 6.1 : Create a scatter plot to visualize the relationship between study Hours and final marks.

```
# plot scatter graph of study hours against final marks
plt.figure(figsize=(8, 6))
plt.scatter(df['study_hours'], df['Final_marks'])
plt.xlabel('Study Hours')
plt.ylabel('Final Marks')
plt.title('Scatter Plot of Study Hours vs Final Marks')
plt.grid(True)
plt.show()
```



Task 6.2: Display the graph.

```
# Displaying the graph
plt.figure(figsize=(8, 6))
plt.scatter(df['study_hours'], df['Final_marks'])
plt.xlabel('Study Hours')
plt.ylabel('Final Marks')
plt.title('Scatter Plot of Study Hours vs Final Marks')
plt.grid(True)
plt.show()
```



Step 7: Calculate covariance.

Task 7.1: Calculate covariance between study hours and final marks.

```
# calculate covariance between study hours and final marks
covariance = df['study_hours'].cov(df['Final_marks'])
print(covariance)
```

```
44.14285714285714
```

Task 7.2: Display the covariance.

```
# calculate  pairwise covariance of all variables
df.cov()
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	2.000000e+01	2.857143	-1.015061e-15	8.857143	10.214286
study_hours	2.857143e+00	6.666667	3.259524e+01	36.000000	44.142857
attendance	-1.015061e-15	32.595238	1.882095e+02	195.328571	239.685714
assignment_score	8.857143e+00	36.000000	1.953286e+02	210.257143	257.328571
Final_marks	1.021429e+01	44.142857	2.396857e+02	257.328571	315.400000

Step 8: Calculate Covariance Matrix.

Task 8.1: Select the numerical columns for covariance analysis.

```
# filter and select only numeric columns
numerical_columns = df.select_dtypes(include=[np.number])
numerical_columns
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
0	101	2	65	55	50
1	102	3	70	60	55
2	103	4	75	65	62
3	104	5	80	70	68
4	105	6	82	72	72
5	106	7	85	78	78
6	107	8	90	82	82
7	108	9	92	88	90
8	109	3	68	62	58
9	110	4	74	67	63
10	111	5	78	71	69
11	112	6	84	76	75
12	113	7	88	80	81
13	114	10	95	95	98
14	115	1	40	35	25

Next steps: Generate code with `numerical_columns` New interactive sheet

Task 8.2: Calculate the covariance Matrix.

```
# calculate covariance matrix for numeric columns
covariance_matrix = numerical_columns.cov()
covariance_matrix
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	2.000000e+01	2.857143	-1.015061e-15	8.857143	10.214286
study_hours	2.857143e+00	6.666667	3.259524e+01	36.000000	44.142857
attendance	-1.015061e-15	32.595238	1.882095e+02	195.328571	239.685714
assignment_score	8.857143e+00	36.000000	1.953286e+02	210.257143	257.328571
Final_marks	1.021429e+01	44.142857	2.396857e+02	257.328571	315.400000

Next steps: Generate code with `covariance_matrix` New interactive sheet

Task 8.3: Display the covariance Matrix.

```
# calculate covariance matrix for the whole dataframe
covariance_matrix = df.cov()
covariance_matrix
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
student_Id	2.000000e+01	2.857143	-1.015061e-15	8.857143	10.214286
study_hours	2.857143e+00	6.666667	3.259524e+01	36.000000	44.142857
attendance	-1.015061e-15	32.595238	1.882095e+02	195.328571	239.685714
assignment_score	8.857143e+00	36.000000	1.953286e+02	210.257143	257.328571
Final_marks	1.021429e+01	44.142857	2.396857e+02	257.328571	315.400000

Next steps: Generate code with `covariance_matrix` New interactive sheet

Step 9: Detect Outliers using IOR Select the final marks colums for outliers analysis.

```
# calculate interquartile range bounds to identify outliers
final_marks = df['Final_marks']
Q1 = final_marks.quantile(0.25)
Q3 = final_marks.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)
print("Lower Bound =", lower_bound)
print("Upper Bound =", upper_bound)
outliers = df[
    (df['Final_marks'] < lower_bound) |
    (df['Final_marks'] > upper_bound)
]
print("\nOutliers:")
print(outliers[['student_Id', 'Final_marks']])
```

```
Q1 = 60.0
Q3 = 79.5
IQR = 19.5
Lower Bound = 30.75
Upper Bound = 108.75

Outliers:
    student_Id  Final_marks
14         115           25
```

Step 10: Identify Outliers.

```
# filter dataframe to locate outlier records
outliers = df[
    (df["Final_marks"] < lower_bound) |
    (df["Final_marks"] > upper_bound)
]
print("\nOutliers:")
print(outliers[["student_Id", "Final_marks"]])
print("\nNumber of Outliers:", len(outliers))
```

```
Outliers:
    student_Id  Final_marks
14         115           25

Number of Outliers: 1
```

Task 10.1 : Select records where final marks are below the lowers bound or above the upper bound.

```
# retrieve records with final marks outside normal range bounds
outliers = df[
    (df["Final_marks"] < lower_bound) |
    (df["Final_marks"] > upper_bound)
]
print("Records with Final_marks outside the bounds:")
print(outliers)
```

```
Records with Final_marks outside the bounds:
    student_Id  study_hours  attendance  assignment_score  Final_marks
14         115            1          40                35           25
```

Task 10.2 : Display the detected outlier records.

```
# print identified outlier records
print("Detected Outlier Records:")
print(outliers)
```

```
Detected Outlier Records:
    student_Id  study_hours  attendance  assignment_score  Final_marks
14         115            1          40                35           25
```

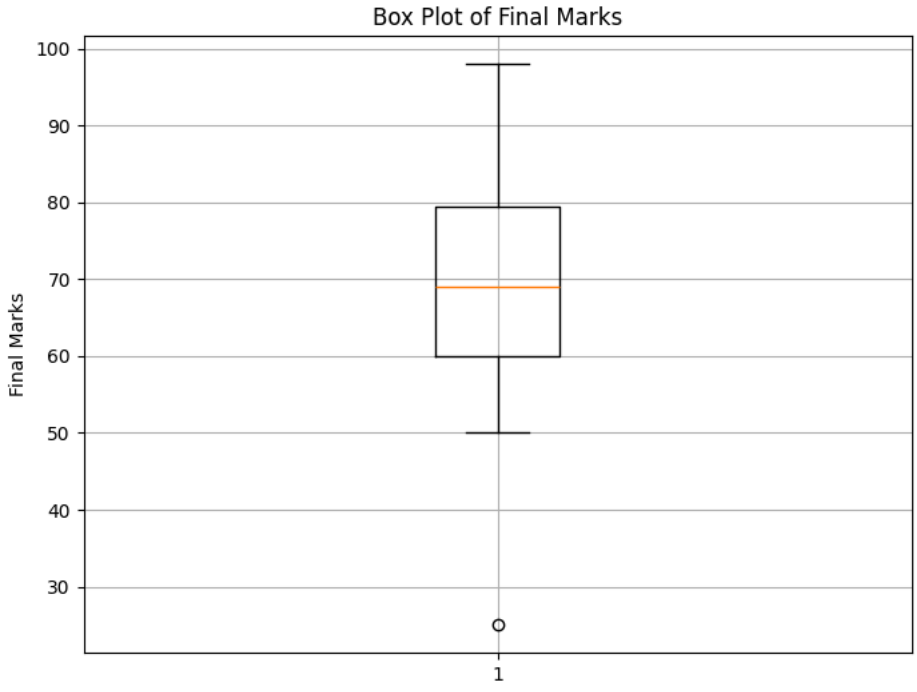
Step 11: Count Outliers.

```
# count total number of outlier records detected
count_outliers = len(outliers)
print(count_outliers)
```

```
1
```

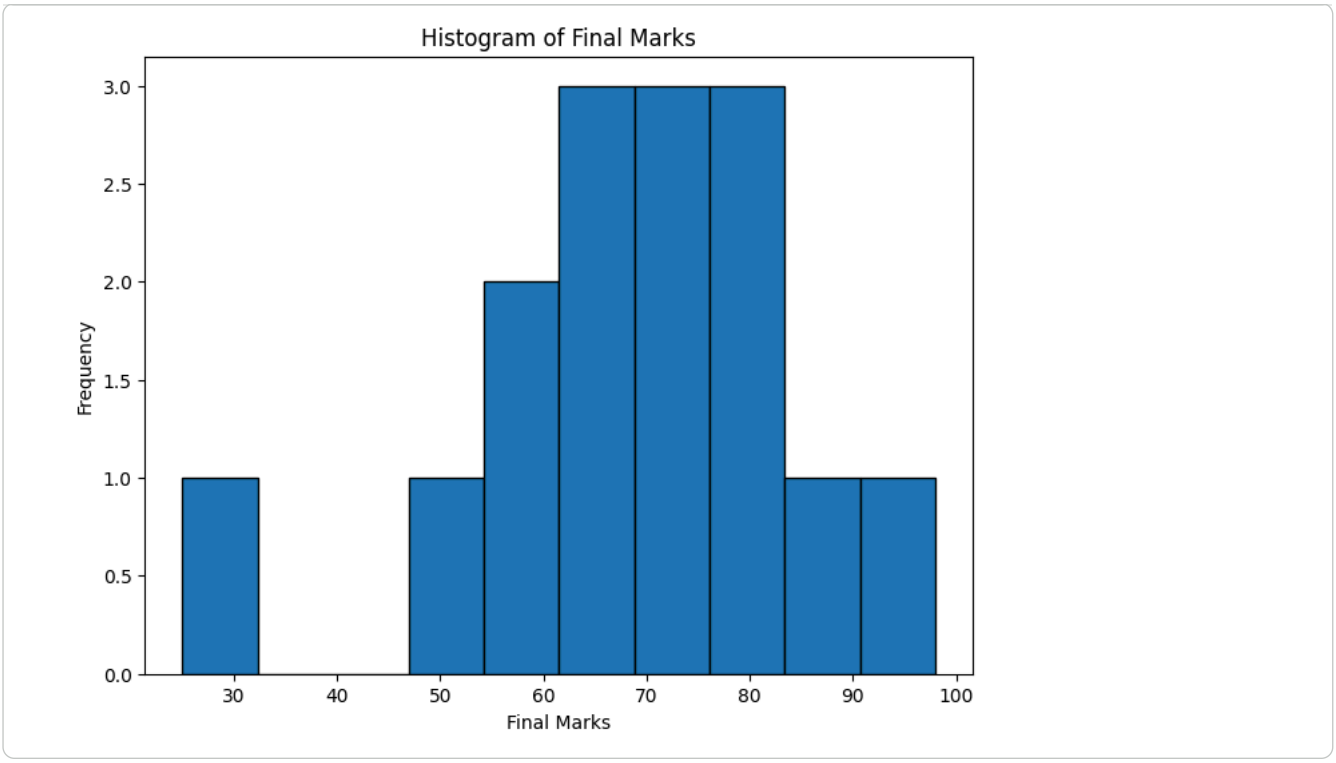
Step 12: Visualize Outliers using Box Plot.

```
# Create box plot of final marks to visualize outliers
plt.figure(figsize=(8, 6))
plt.boxplot(df['Final_marks'])
plt.ylabel('Final Marks')
plt.title('Box Plot of Final Marks')
plt.grid(True)
plt.show()
```



Step 13: Create a Histogram to observe the distribution of Final Marks.

```
# Create histogram to display distribution of final marks
plt.figure(figsize=(8, 6))
plt.hist(df['Final_marks'], bins=10, edgecolor='black')
plt.xlabel('Final Marks')
plt.ylabel('Frequency')
plt.title('Histogram of Final Marks')
plt.show()
```



Step 14: Analyze distribution using Skewness.

Task 14.1: Calculate the skewness of final marks.

```
# calculate skewness of final marks distribution
skewness = df['Final_marks'].skew()
print("Skewness of Final_marks:", skewness)
```

```
Skewness of Final_marks: -0.7383380071904277
```

Task 14.2: Display the Skewness Value.

```
# display calculated skewness value for final marks
skewness = df['Final_marks'].skew()
print("Skewness Value:", skewness)
```

```
Skewness Value: -0.7383380071904277
```

Step 15: Calculate Kurtosis.

Task 15.1: Calculate the kurtosis of final marks.

```
# calculate kurtosis of final marks distribution
kurtosis = df['Final_marks'].kurtosis()
print("Kurtosis of Final_marks:", kurtosis)
```

```
Kurtosis of Final_marks: 1.4936683306648835
```

Task 15.2:Display the kurtosis value.

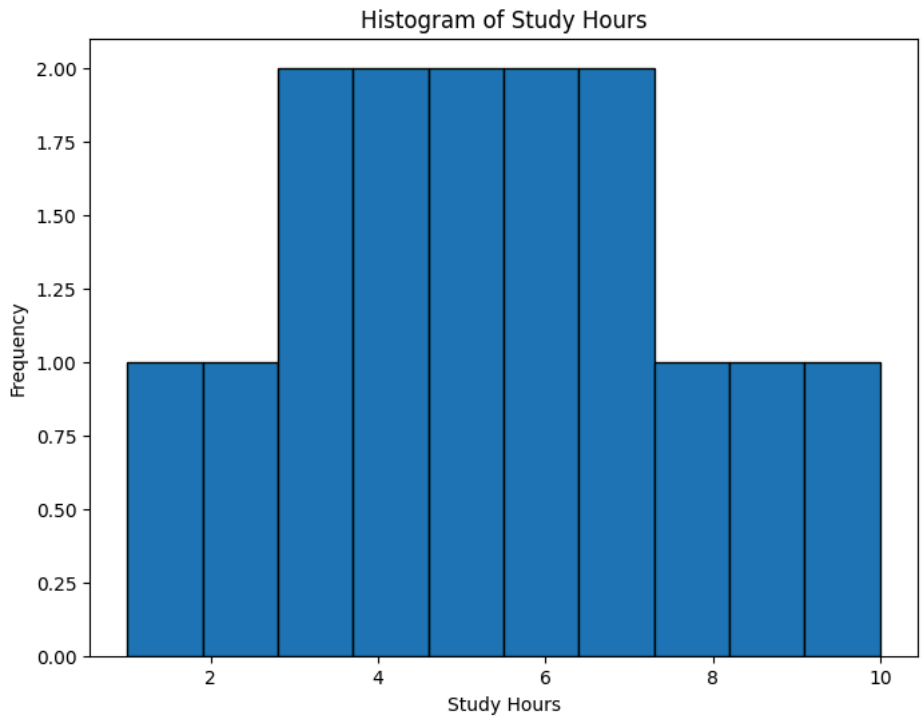
```
# display calculated kurtosis value for final marks
kurtosis = df['Final_marks'].kurtosis()
print("Kurtosis Value:", kurtosis)
```

```
Kurtosis Value: 1.4936683306648835
```

Step 16: Distribution of study hours and create a histogram for study hours.

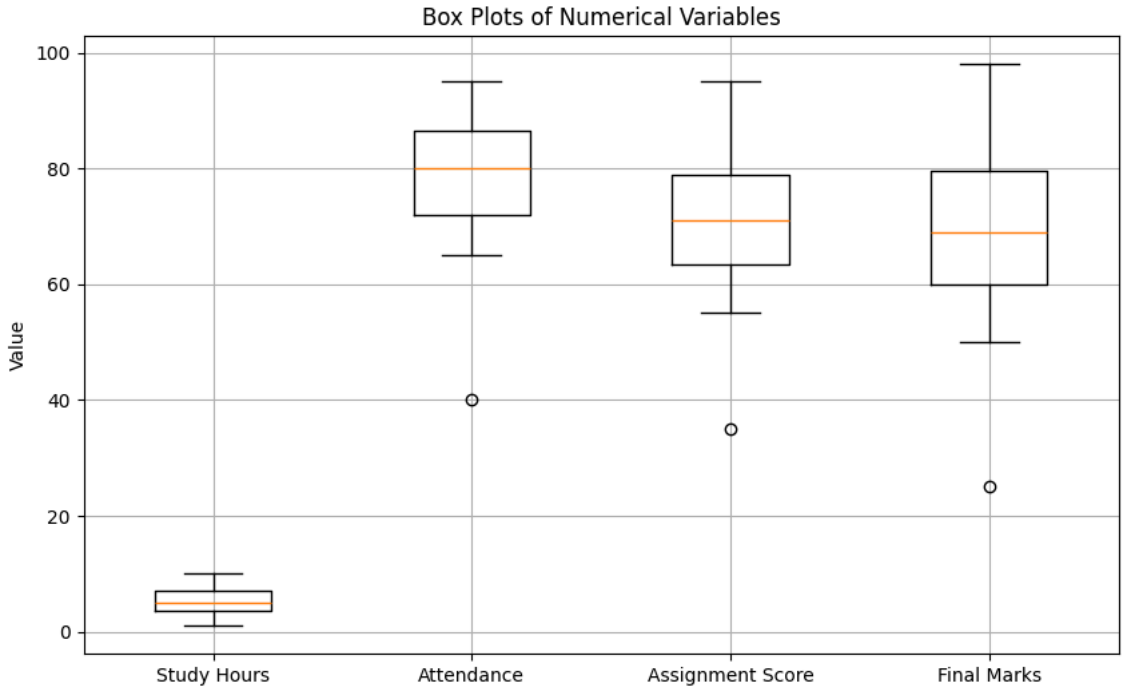
```
# render histogram to analyze distribution of study hours
plt.figure(figsize=(8, 6))
plt.hist(df['study_hours'], bins=10, edgecolor='black')
plt.xlabel('Study Hours')
plt.ylabel('Frequency')
```

```
plt.title('Histogram of Study Hours')
plt.show()
```



Step 17: Compare variables using box plotscreate box plots for multiple numerical variables.

```
# render box plots to compare distributions of multiple numeric features
plt.figure(figsize=(10, 6))
plt.boxplot([df['study_hours'], df['attendance'], df['assignment_score'], df['Final_marks']])
plt.xticks([1, 2, 3, 4], ['Study Hours', 'Attendance', 'Assignment Score', 'Final Marks'])
plt.ylabel('Value')
plt.title('Box Plots of Numerical Variables')
plt.grid(True)
plt.show()
```



Step 18: Generate a Statistical Summary.

```
# display computed descriptive statistical summary
statistical_summary = df.describe()
statistical_summary
```

	student_Id	study_hours	attendance	assignment_score	Final_marks
count	15.000000	15.000000	15.000000	15.000000	15.000000
mean	108.000000	5.333333	77.733333	70.400000	68.400000
std	4.472136	2.581989	13.718948	14.500246	17.759504
min	101.000000	1.000000	40.000000	35.000000	25.000000
25%	104.500000	3.500000	72.000000	63.500000	60.000000
50%	108.000000	5.000000	80.000000	71.000000	69.000000
75%	111.500000	7.000000	86.500000	79.000000	79.500000
max	115.000000	10.000000	95.000000	95.000000	98.000000

Next steps: Generate code with `statistical_summary` New interactive sheet

Step 19: Save the Analysis Dataset.

```
# export cleaned dataframe to local csv file
df.to_csv('analysis_dataset.csv', index=False)
print("Analysis dataset saved successfully.")
```